

Product Batch - Task Submission - 31.08.2026

ROLL NO : 2303A51027

NAME : Aravind Reddy

1. Print the boundary nodes of a binary tree.

```
main.py Visualize Run ▶

26
27 # =====
28 # 1. PRINT BOUNDARY NODES
29 # =====
30
31 def is_leaf(node):
32     return node is not None and node.left is None and node.right is None
33
34
35 def print_left_boundary(root):
36     node = root.left
37
38     while node:
39         if not is_leaf(node):
40             print(node.data, end=" ")
41
42         if node.left:
43             node = node.left
44         else:
45             node = node.right
46
47
48 def print_leaves(root):
49     if root is None:
50         return
51
52     print_leaves(root.left)
53
54     if is_leaf(root):
55         print(root.data, end=" ")
56
57     print_leaves(root.right)
```

main.py

VisualizeRun ▶

```
59
60 def print_right_boundary(root):
61     node = root.right
62     stack = []
63
64     while node:
65         if not is_leaf(node):
66             stack.append(node.data)
67
68         if node.right:
69             node = node.right
70         else:
71             node = node.left
72
73     while stack:
74         print(stack.pop(), end=" ")
75
76
77 def boundary_traversal(root):
78     if root is None:
79         return
80
81     print("Boundary Nodes:", end=" ")
82
83     # Root
84     print(root.data, end=" ")
85
86     # Left boundary
87     print_left_boundary(root)
88
89     # Leaf nodes
90     print_leaves(root)
91
```

main.py

VisualizeRun ▶

```
86     # Left boundary
87     print_left_boundary(root)
88
89     # Leaf nodes
90     print_leaves(root)
91
92     # Right boundary
93     print_right_boundary(root)
94
95     print()
96
```

2. Print the left view of a binary tree.

```
98 # =====
99 # 2. PRINT LEFT VIEW
100 # =====
101
102 def left_view(root):
103     if root is None:
104         return []
105
106     result = []
107     queue = [root]
108
109     while queue:
110
111         level_size = len(queue)
112
113         for i in range(level_size):
114
115             node = queue.pop(0)
116
117             # First node of every level
118             if i == 0:
119                 result.append(node.data)
120
121             if node.left:
122                 queue.append(node.left)
123
124             if node.right:
125                 queue.append(node.right)
126
127     return result
128
129
130 # =====
```

3. Print the right view of a binary tree.

main.pyVisualizeRun ▶

```
129
130 # =====
131 # 3. PRINT RIGHT VIEW
132 # =====
133
134 def right_view(root):
135     if root is None:
136         return []
137
138     result = []
139     queue = [root]
140
141     while queue:
142
143         level_size = len(queue)
144
145         for i in range(level_size):
146
147             node = queue.pop(0)
148
149             # Last node of every level
150             if i == level_size - 1:
151                 result.append(node.data)
152
153             if node.left:
154                 queue.append(node.left)
155
156             if node.right:
157                 queue.append(node.right)
158
159     return result
160
```

4. Find the lowest common ancestor (LCA) of two nodes in a binary tree.

main.py

Visualize

Run ▶

```
161
162 # =====
163 # 4. LOWEST COMMON ANCESTOR (LCA)
164 # =====
165
166 def find_lca(root, n1, n2):
167
168     if root is None:
169         return None
170
171     # If root is one of the nodes
172     if root.data == n1 or root.data == n2:
173         return root
174
175     left_lca = find_lca(root.left, n1, n2)
176     right_lca = find_lca(root.right, n1, n2)
177
178     # Nodes are present in different subtrees
179     if left_lca and right_lca:
180         return root
181
182     # Return whichever side contains a node
183     if left_lca:
184         return left_lca
185
186     return right_lca
187
188
```

5. Find the path from the root to a given node in a binary tree.

main.py

VisualizeRun ▶

```
189 # =====
190 # 5. PATH FROM ROOT TO GIVEN NODE
191 # =====
192
193 def find_path(root, target, path):
194
195     if root is None:
196         return False
197
198     # Add current node
199     path.append(root.data)
200
201     # Target found
202     if root.data == target:
203         return True
204
205     # Search left or right subtree
206     if find_path(root.left, target, path) or find_path(root.right, target, path):
207         return True
208
209     # Remove node if target not found
210     path.pop()
211
212     return False
213
214
215 def root_to_node_path(root, target):
216
217     path = []
218
219     if find_path(root, target, path):
220         return path
221
```

main.py

VisualizeRun ▶

```
205 # Search left or right subtree
206 if find_path(root.left, target, path) or find_path(root.right, target, path):
207     return True
208
209 # Remove node if target not found
210 path.pop()
211
212 return False
213
214
215 def root_to_node_path(root, target):
216
217     path = []
218
219     if find_path(root, target, path):
220         return path
221
222     return []
223
```

6. Construct a binary tree from inorder and preorder traversal.

```
main.py Visualize Run ▶
262
263
264 # =====
265 # TRAVERSAL FUNCTIONS FOR QUESTION 6
266 # =====
267
268 def inorder_traversal(root):
269
270     if root is None:
271         return []
272
273     return inorder_traversal(root.left) + [root.data] + inorder_traversal(root.right)
274
275
276 def preorder_traversal(root):
277
278     if root is None:
279         return []
280
281     return [root.data] + preorder_traversal(root.left) + preorder_traversal(root.right)
282
283
```

OUTPUT

```
=====
                BINARY TREE QUESTIONS
=====

1. Boundary Nodes
Boundary Nodes: 1 2 4 8 9 6 7 3

2. Left View
Left View: [1, 2, 4, 8]

3. Right View
Right View: [1, 3, 7, 9]

4. Lowest Common Ancestor
LCA of 8 and 9 = 5

5. Path from Root to Node
Path to 9 : [1, 2, 5, 9]

6. Construct Tree from Inorder and Preorder
Given Inorder : [4, 2, 8, 5, 9, 1, 6, 3, 7]
Given Preorder: [1, 2, 4, 5, 8, 9, 3, 6, 7]
Constructed Tree Inorder : [4, 2, 8, 5, 9, 1, 6, 3, 7]
Constructed Tree Preorder: [1, 2, 4, 5, 8, 9, 3, 6, 7]

=== Code Execution Successful ===
```